Workshop Snowflake per Mediaset

Guida per i Partecipanti

Informazioni Workshop

Durata	2 ore
Livello	Introduttivo/Intermedio
Prerequisiti	Account Snowflake attivo, conoscenze base SQL
Settore	TV Broadcasting

Agenda

Orario	Modulo	Durata
00:00 - 00:35	Modulo 1: Setup e Basi Snowflake	35 min
00:35 - 00:55	Modulo 2: Zero Copy Cloning e Time Travel	20 min
00:55 - 01:25	Modulo 3: Row & Column Level Security	30 min
01:25 - 02:00	Modulo 4: Data Pipelines (Dynamic Tables, Streams & Tasks)	35 min

---

Modulo 1: Setup e Basi Snowflake (35 min)

> **Snowflake** è una piattaforma dati cloud-native che separa storage e compute. I **Database** organizzano i dati in **Schema**, mentre i **Warehouse** forniscono risorse di calcolo elastiche on-demand. I **Ruoli** controllano l'accesso seguendo il principio del least privilege.

Obiettivi di Apprendimento

- Creare e configurare database, schema e warehouse
- Comprendere il modello di ruoli e permessi
- Eseguire query SQL di base
- Caricare e interrogare dati

Istruzioni Passo-Passo

Step 1.1: Accesso a Snowflake

1. Apri il browser e vai all'URL del tuo account Snowflake
2. Effettua il login con le credenziali fornite
3. Nel menu laterale, vai su **Projects** > **Workspaces**

Workspaces è l'ambiente di lavoro unificato di Snowsight che combina fogli SQL, notebook Python e file in un unico spazio collaborativo. Permette di organizzare query, notebook e risorse in cartelle condivisibili con il team, sostituendo la precedente sezione "Worksheets".

Step 1.2: Creazione Database e Schema

```
-- Usa SYSADMIN per creare oggetti (best practice)
USE ROLE SYSADMIN;

-- Crea il database per il workshop
CREATE OR REPLACE DATABASE MEDIASET_LAB;

-- Crea gli schema necessari
CREATE OR REPLACE SCHEMA MEDIASET_LAB.RAW;
CREATE OR REPLACE SCHEMA MEDIASET_LAB.ANALYTICS;
CREATE OR REPLACE SCHEMA MEDIASET_LAB.SECURITY;
```

Verifica: Esegui `SHOW SCHEMAS IN DATABASE MEDIASET_LAB;`

Step 1.3: Creazione Warehouse

```
-- Usa SYSADMIN per creare warehouse (best practice)
USE ROLE SYSADMIN;

CREATE OR REPLACE WAREHOUSE MEDIASET_WH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 300
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;
```

Nota: Il warehouse si attiverà automaticamente alla prima query.

Step 1.4: Creazione Ruoli

```
-- Usa SECURITYADMIN per gestire ruoli (best practice)
USE ROLE SECURITYADMIN;

-- Crea i ruoli custom
CREATE OR REPLACE ROLE MEDIASET_ADMIN;
CREATE OR REPLACE ROLE MEDIASET_ANALYST;
CREATE OR REPLACE ROLE MEDIASET_MARKETING;
CREATE OR REPLACE ROLE MEDIASET_REGIONALE_NORD;
CREATE OR REPLACE ROLE MEDIASET_REGIONALE_SUD;

-- BEST PRACTICE: I ruoli custom devono riportare a SYSADMIN
-- Questo garantisce che SYSADMIN possa gestire tutti gli oggetti
GRANT ROLE MEDIASET_ADMIN TO ROLE SYSADMIN;
GRANT ROLE MEDIASET_ANALYST TO ROLE SYSADMIN;
GRANT ROLE MEDIASET_MARKETING TO ROLE SYSADMIN;
GRANT ROLE MEDIASET_REGIONALE_NORD TO ROLE SYSADMIN;
GRANT ROLE MEDIASET_REGIONALE_SUD TO ROLE SYSADMIN;

-- Gerarchia interna dei ruoli Mediaset
GRANT ROLE MEDIASET_ANALYST TO ROLE MEDIASET_ADMIN;
GRANT ROLE MEDIASET_MARKETING TO ROLE MEDIASET_ADMIN;
GRANT ROLE MEDIASET_REGIONALE_NORD TO ROLE MEDIASET_ANALYST;
GRANT ROLE MEDIASET_REGIONALE_SUD TO ROLE MEDIASET_ANALYST;
```

Best Practice: Tutti i ruoli custom devono essere assegnati a SYSADMIN per mantenere la gerarchia standard di Snowflake.

Step 1.5: Creazione Tabelle

Esegui le sezioni 1-4 dello script `01_setup_data.sql` per creare database, schema, warehouse, ruoli e tabelle.

Step 1.6: Caricamento Dati da CSV tramite Snowsight

I dati sono forniti come file CSV nella cartella `data/`. Per caricare ogni tabella:

1. Nel menu laterale, clicca su **Catalog** > **Database Explorer**
2. Naviga fino a **MEDIASET_LAB** > **RAW**
3. Clicca sulla tabella da popolare (es. `PROGRAMMI_TV`)
4. In alto a destra, clicca su **Load Data**
5. Seleziona il warehouse **MEDIASET_WH**
6. Clicca su **Browse** e seleziona il file CSV corrispondente dalla cartella `data/`
7. Nelle opzioni del File Format:
8. **File Type**: CSV
9. **Header**: seleziona "First line contains header"
10. **Field delimiter**: virgola (`,`)
11. **Field optionally enclosed by**: doppio apice (`"`)
12. Clicca su **Load** per avviare il caricamento

Ripeti per tutte le tabelle nell'ordine indicato:

#	Tabella	File CSV	Righe attese
1	PROGRAMMI_TV	`data/PROGRAMMI_TV.csv`	20
2	PALINSESTO	`data/PALINSESTO.csv`	600
3	ASCOLTI	`data/ASCOLTI.csv`	5000
4	ABBONATI	`data/ABBONATI.csv`	20
5	CONTENUTI_DESCRIZIONI	`data/CONTENUTI_DESCRIZIONI.csv`	20
6	CONTRATTI_PUBBLICITARI	`data/CONTRATTI_PUBBLICITARI.csv`	15
7	FEEDBACK_SOCIAL	`data/FEEDBACK_SOCIAL.csv`	20

Nota: Carica `PROGRAMMI_TV` per prima, poiché le altre tabelle contengono riferimenti al campo `programma_id`.

Step 1.7: Verifica Caricamento

```
-- Verifica che tutte le tabelle siano state caricate correttamente
SELECT 'PROGRAMMI_TV' as tabella, COUNT(*) as righe FROM
MEDIASET_LAB.RAW.PROGRAMMI_TV
UNION ALL SELECT 'PALINSESTO', COUNT(*) FROM MEDIASET_LAB.RAW.PALINSESTO
UNION ALL SELECT 'ASCOLTI', COUNT(*) FROM MEDIASET_LAB.RAW.ASCOLTI
UNION ALL SELECT 'ABBONATI', COUNT(*) FROM MEDIASET_LAB.RAW.ABBONATI
UNION ALL SELECT 'CONTENUTI_DESCRIZIONI', COUNT(*) FROM
MEDIASET_LAB.RAW.CONTENUTI_DESCRIZIONI
UNION ALL SELECT 'CONTRATTI_PUBBLICITARI', COUNT(*) FROM
MEDIASET_LAB.RAW.CONTRATTI_PUBBLICITARI
UNION ALL SELECT 'FEEDBACK_SOCIAL', COUNT(*) FROM
MEDIASET_LAB.RAW.FEEDBACK_SOCIAL
ORDER BY tabella;
```

Step 1.8: Query di Verifica

```
-- Verifica le tabelle create
SHOW TABLES IN SCHEMA MEDIASET_LAB.RAW;

-- Query sui programmi TV
SELECT titolo, genere, canale, costo_episodio_eur
FROM MEDIASET_LAB.RAW.PROGRAMMI_TV
ORDER BY costo_episodio_eur DESC
LIMIT 5;

-- Analisi ascolti per regione
SELECT
    regione,
    AVG(share_percentuale) as share_medio,
    SUM(telespettatori) as telespettatori_totali
FROM MEDIASET_LAB.RAW.ASCOLTI
GROUP BY regione
ORDER BY share_medio DESC;

-- Elasticità del Warehouse: scala da XSMALL a MEDIUM
ALTER WAREHOUSE MEDIASET_WH SET WAREHOUSE_SIZE = 'MEDIUM';
```

Nota: Osserva il tempo di esecuzione del comando `ALTER WAREHOUSE`: il ridimensionamento è quasi istantaneo. Snowflake scala le risorse di calcolo in pochi secondi, senza interruzioni per le query in corso. Questa è una delle caratteristiche chiave dell'architettura cloud-native di Snowflake.

```
-- Torna alla size precedente per risparmiare crediti
ALTER WAREHOUSE MEDIASET_WH SET WAREHOUSE_SIZE = 'XSMALL';
```

Best Practice: Usa la size minima necessaria per il carico di lavoro. Scala verso l'alto solo quando servono prestazioni maggiori, e ricorda di tornare alla size originale al termine.

Step 1.9: Caricamento Dati da Stage S3 Pubblico

Oltre al caricamento manuale via Snowsight, Snowflake permette di caricare dati direttamente da bucket S3 pubblici utilizzando uno **Stage esterno**. Questo approccio è ideale per automatizzare l'ingestion e lavorare con dataset di grandi dimensioni senza scaricarli in locale.

In questo esempio utilizziamo il dataset **Citibike Trips**, ospitato nel bucket S3 pubblico `s3://snowflake-workshop-lab/citibike-trips-csv/` (accessibile senza credenziali AWS).

```
USE ROLE SYSADMIN;
USE SCHEMA MEDIASET_LAB.RAW;
USE WAREHOUSE MEDIASET_WH;

-- Passo 1: Creare un file format per CSV compressi (gzip)
CREATE OR REPLACE FILE FORMAT CSV_GZIP_FORMAT
    TYPE = 'CSV'
    COMPRESSION = 'GZIP'
    FIELD_DELIMITER = ','
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER = 0;

-- Passo 2: Creare uno stage esterno che punta al bucket S3 pubblico
CREATE OR REPLACE STAGE CITIBIKE_STAGE
    URL = 's3://snowflake-workshop-lab/citibike-trips-csv/'
    FILE_FORMAT = CSV_GZIP_FORMAT;

-- Passo 3: Verificare che lo stage sia raggiungibile
LIST @CITIBIKE_STAGE;

-- Passo 4: Creare la tabella di destinazione
CREATE OR REPLACE TABLE CITIBIKE_TRIPS (
    TRIP_ID             INT,
    STARTTIME           TIMESTAMP,
    STOPTIME            TIMESTAMP,
    TRIPDURATION_MIN    INT,
    START_STATION_ID    INT,
    END_STATION_ID      INT,
    BIKEID              VARCHAR(20),
    BIKE_TYPE           VARCHAR(20),
    USER_ID             INT,
    USER_NAME           VARCHAR(100),
    BIRTH_DATE          DATE,
    GENDER              VARCHAR(10),
    USER_TYPE           VARCHAR(50),
    PAYMENT_METHOD      VARCHAR(20),
    PAYMENT_PROVIDER    VARCHAR(20)
);

-- Passo 5: Caricare i dati con COPY INTO (un singolo file per velocità)
COPY INTO CITIBIKE_TRIPS
    FROM @CITIBIKE_STAGE
    PATTERN = '.*data_0_1_0.csv.gz'
    ON_ERROR = 'CONTINUE';

-- Passo 6: Verificare il caricamento
SELECT COUNT(*) AS righe_caricate FROM CITIBIKE_TRIPS;
SELECT * FROM CITIBIKE_TRIPS LIMIT 5;
```

Nota: Il comando `LIST` mostra i file disponibili nello stage. Il `PATTERN` nel `COPY INTO` limita il caricamento a un singolo file per velocizzare l'esempio. Rimuovendo il `PATTERN` si caricano tutti i file presenti.

Esercizio Pratico 1

Scrivi una query che mostri i top 5 programmi per share medio, includendo genere e canale.

Modulo 2: Zero Copy Cloning e Time Travel (20 min)

> Il **Zero Copy Cloning** crea una copia istantanea di tabelle, schema o database senza duplicare i dati fisici — lo storage viene condiviso fino a quando una delle copie viene modificata. Il **Time Travel** permette di accedere ai dati storici (fino a 90 giorni) per recuperare dati cancellati, confrontare versioni e fare audit.

Obiettivi di Apprendimento

- Clonare tabelle e schema senza costi di storage aggiuntivi
- Utilizzare Time Travel per recuperare dati modificati o cancellati
- Comprendere UNDROP per ripristinare oggetti eliminati

Istruzioni Passo-Passo

Step 2.1: Zero Copy Clone di una Tabella

```
USE ROLE SYSADMIN;
USE SCHEMA MEDIASET_LAB.RAW;

-- Clona la tabella PROGRAMMI_TV (istantaneo, zero storage aggiuntivo)
CREATE OR REPLACE TABLE PROGRAMMI_TV_CLONE CLONE PROGRAMMI_TV;

-- Verifica: il clone contiene gli stessi dati
SELECT COUNT(*) as righe_originale FROM PROGRAMMI_TV;
SELECT COUNT(*) as righe_clone FROM PROGRAMMI_TV_CLONE;
```

Nota: Il clone è un oggetto indipendente. Le modifiche al clone NON influenzano l'originale e viceversa.

Step 2.2: Modifica Indipendente del Clone

```
-- Modifica il clone: aggiorna un record
UPDATE PROGRAMMI_TV_CLONE
SET costo_episodio_eur = 999999
WHERE titolo = 'Grande Fratello VIP';

-- Verifica: il clone è stato modificato
SELECT titolo, costo_episodio_eur
FROM PROGRAMMI_TV_CLONE
WHERE titolo = 'Grande Fratello VIP';

-- Verifica: l'originale è rimasto invariato
SELECT titolo, costo_episodio_eur
FROM PROGRAMMI_TV
WHERE titolo = 'Grande Fratello VIP';
```

Step 2.3: Time Travel - Recupero Dati Cancellati

```
-- Salva il conteggio attuale
SELECT COUNT(*) as righe_prima FROM PROGRAMMI_TV;

-- Cancella alcuni record
DELETE FROM PROGRAMMI_TV WHERE genere = 'Reality';

-- Verifica la cancellazione
SELECT COUNT(*) as righe_dopo FROM PROGRAMMI_TV;

-- Time Travel: vedi i dati com'erano 1 minuto fa
SELECT COUNT(*) as righe_nel_passato
FROM PROGRAMMI_TV AT (OFFSET => -60);

-- Ripristina i dati cancellati usando Time Travel
INSERT INTO PROGRAMMI_TV
SELECT * FROM PROGRAMMI_TV AT (OFFSET => -60)
WHERE genere = 'Reality';

-- Verifica il ripristino
SELECT COUNT(*) as righe_ripristinate FROM PROGRAMMI_TV;
```

Step 2.4: UNDROP - Ripristino di una Tabella Eliminata

```
-- Elimina la tabella clone
DROP TABLE PROGRAMMI_TV_CLONE;

-- Prova a interrogarla (errore!)
-- SELECT * FROM PROGRAMMI_TV_CLONE;

-- Ripristina con UNDROP
UNDROP TABLE PROGRAMMI_TV_CLONE;

-- Verifica: la tabella è tornata
SELECT COUNT(*) FROM PROGRAMMI_TV_CLONE;

-- Pulizia: elimina definitivamente il clone
DROP TABLE PROGRAMMI_TV_CLONE;
```

Esercizio Pratico 2

Clona l'intero schema `RAW` in un nuovo schema `RAW_BACKUP` usando `CREATE SCHEMA RAW_BACKUP CLONE RAW` e verifica che tutte le tabelle siano presenti.

Modulo 3: Row & Column Level Security (30 min)

> Le **Masking Policy** nascondono o offuscano dati sensibili (PII) a livello di colonna in base al ruolo dell'utente. Le **Row Access Policy** filtrano automaticamente le righe visibili, garantendo che ogni utente veda solo i dati di sua competenza senza modificare le query.

Obiettivi di Apprendimento

- Implementare masking policy per proteggere dati sensibili
- Creare row access policy per filtrare dati per ruolo
- Testare le policy con ruoli diversi

Istruzioni Passo-Passo

Step 3.1: Tag-Based Masking Policy per Email

I **Tag** in Snowflake sono metadati che si associano a oggetti e colonne per classificarli. Sono fondamentali sia per la **governance** (identificare dati sensibili come PII, PHI, PCI) sia per il **cost allocation** (tracciare l'utilizzo delle risorse per business unit o progetto tramite i tag sui warehouse). Associando una masking policy a un tag, ogni colonna taggata viene automaticamente protetta — senza doverla configurare una per una.

```
USE ROLE ACCOUNTADMIN;
USE SCHEMA MEDIASET_LAB.SECURITY;

-- Passo 1: Creare un tag per classificare le colonne contenenti email
CREATE OR REPLACE TAG MEDIASET_LAB.SECURITY.PII_TYPE
    ALLOWED_VALUES = 'EMAIL', 'TELEFONO', 'NOME', 'INDIRIZZO'
    COMMENT = 'Classifica il tipo di dato personale (PII) contenuto nella
colonna';

-- Passo 2: Creare la masking policy per email
CREATE OR REPLACE MASKING POLICY email_mask AS (val STRING) RETURNS
STRING ->
    CASE
        WHEN CURRENT_ROLE() IN ('MEDIASET_ADMIN', 'ACCOUNTADMIN') THEN
val
        WHEN CURRENT_ROLE() = 'MEDIASET_MARKETING' THEN
            REGEXP_REPLACE(val, '(.{2})(.*)(@.*)', '\\1***\\3')
        ELSE '***RISERVATO***'
    END;

-- Passo 3: Associare la masking policy al tag
-- Ogni colonna che riceverà il tag PII_TYPE = 'EMAIL' sarà
automaticamente mascherata
ALTER TAG MEDIASET_LAB.SECURITY.PII_TYPE
    SET MASKING POLICY email_mask;

-- Passo 4: Assegnare il tag alla colonna email della tabella ABBONATI
ALTER TABLE MEDIASET_LAB.RAW.ABBONATI
    MODIFY COLUMN email SET TAG MEDIASET_LAB.SECURITY.PII_TYPE = 'EMAIL';
```

Vantaggi del Tag-Based Masking:

- **Scalabilità**: se domani aggiungi una nuova tabella con una colonna email, basta assegnarle il tag e la policy si applica automaticamente.
- **Governance centralizzata**: tutti i dati PII sono catalogati e tracciabili tramite i tag.

- **Cost allocation**: i tag applicati ai warehouse permettono di attribuire i costi di calcolo a specifici team o progetti (es. `TAG COST_CENTER = 'MARKETING'`).

Step 3.2: Verifica del Tag e della Policy

```
-- Verifica quali tag sono assegnati alla colonna
SELECT * FROM TABLE(INFORMATION_SCHEMA.TAG_REFERENCES(
    'MEDIASET_LAB.RAW.ABBONATI.EMAIL', 'COLUMN'
));

-- Verifica le masking policy attive
SHOW MASKING POLICIES IN SCHEMA MEDIASET_LAB.SECURITY;
```

Step 3.3: Test con Ruoli Diversi

```
-- Test come ADMIN (vede tutto)
USE ROLE MEDIASET_ADMIN;
SELECT nome, cognome, email FROM MEDIASET_LAB.RAW.ABBONATI LIMIT 5;

-- Test come MARKETING (vede parzialmente)
USE ROLE MEDIASET_MARKETING;
SELECT nome, cognome, email FROM MEDIASET_LAB.RAW.ABBONATI LIMIT 5;

-- Test come ANALYST (vede mascherato)
USE ROLE MEDIASET_ANALYST;
SELECT nome, cognome, email FROM MEDIASET_LAB.RAW.ABBONATI LIMIT 5;
```

Step 3.4: Row Access Policy

```
USE ROLE ACCOUNTADMIN;

-- Crea mapping ruolo-regione
CREATE OR REPLACE TABLE MEDIASET_LAB.SECURITY.ROLE_REGION_MAPPING (
    role_name VARCHAR(100),
    regione VARCHAR(50)
);

INSERT INTO MEDIASET_LAB.SECURITY.ROLE_REGION_MAPPING VALUES
('MEDIASET_REGIONALE_NORD', 'Lombardia'),
('MEDIASET_REGIONALE_NORD', 'Piemonte'),
('MEDIASET_REGIONALE_NORD', 'Veneto'),
('MEDIASET_REGIONALE_SUD', 'Campania'),
('MEDIASET_REGIONALE_SUD', 'Sicilia'),
('MEDIASET_REGIONALE_SUD', 'Puglia');

-- Crea la policy
CREATE OR REPLACE ROW ACCESS POLICY region_access_policy
AS (regione_col VARCHAR) RETURNS BOOLEAN ->
    CASE
        WHEN CURRENT_ROLE() IN ('MEDIASET_ADMIN', 'ACCOUNTADMIN',
'MEDIASET_ANALYST') THEN TRUE
        WHEN CURRENT_ROLE() = 'MEDIASET_MARKETING' THEN TRUE
        WHEN EXISTS (
            SELECT 1 FROM MEDIASET_LAB.SECURITY.ROLE_REGION_MAPPING
            WHERE role_name = CURRENT_ROLE() AND regione = regione_col
        ) THEN TRUE
        ELSE FALSE
    END;

-- Applica la policy
ALTER TABLE MEDIASET_LAB.RAW.ASCOLTI
    ADD ROW ACCESS POLICY region_access_policy ON (regione);
```

Step 3.5: Test Row Access Policy

```
-- Come REGIONALE_NORD vedo solo regioni del nord
USE ROLE MEDIASET_REGIONALE_NORD;
SELECT DISTINCT regione FROM MEDIASET_LAB.RAW.ASCOLTI;

-- Come REGIONALE_SUD vedo solo regioni del sud
USE ROLE MEDIASET_REGIONALE_SUD;
SELECT DISTINCT regione FROM MEDIASET_LAB.RAW.ASCOLTI;
```

Step 3.6: Aggregation Policy

Le **Aggregation Policy** impediscono che una query restituisca risultati basati su un numero troppo piccolo di record, proteggendo contro la re-identificazione di individui. Sono utili quando si condividono dati aggregati con ruoli che non devono poter isolare singoli record.

```
USE ROLE ACCOUNTADMIN;
USE SCHEMA MEDIASET_LAB.SECURITY;

-- Creare una aggregation policy che richiede almeno 5 record per gruppo
-- Se una query restituisce un gruppo con meno di 5 righe, il risultato
viene bloccato
CREATE OR REPLACE AGGREGATION POLICY min_aggregation_policy
    AS () RETURNS AGGREGATION_CONSTRAINT ->
    CASE
        WHEN CURRENT_ROLE() IN ('MEDIASET_ADMIN', 'ACCOUNTADMIN')
            THEN NO_AGGREGATION_CONSTRAINT()
        ELSE AGGREGATION_CONSTRAINT(MIN_GROUP_SIZE => 5)
    END;

-- Applicare la policy alla tabella ABBONATI
ALTER TABLE MEDIASET_LAB.RAW.ABBONATI
    SET AGGREGATION POLICY min_aggregation_policy;
```

Test della Aggregation Policy:

```
-- Come ANALYST, le query aggregate funzionano normalmente
USE ROLE MEDIASET_ANALYST;
SELECT regione, COUNT(*) as num_abbonati, AVG(importo_mensile) as
media_importo
FROM MEDIASET_LAB.RAW.ABBONATI
GROUP BY regione;

-- Ma una query senza aggregazione o con gruppi troppo piccoli viene
bloccata
-- Questa query fallirà perché tenta di accedere a righe individuali:
-- SELECT * FROM MEDIASET_LAB.RAW.ABBONATI LIMIT 5;

-- Come ADMIN, nessuna restrizione
USE ROLE MEDIASET_ADMIN;
SELECT * FROM MEDIASET_LAB.RAW.ABBONATI LIMIT 5;
```

Nota: Le aggregation policy sono particolarmente utili in scenari di data sharing, dove si vuole garantire che i consumatori possano solo analizzare dati aggregati senza mai accedere a record individuali.

Esercizio Pratico 3

Crea una masking policy per il campo `telefono` che mostri solo le prime 6 cifre ai ruoli non admin.

Modulo 4: Data Pipelines - Dynamic Tables, Streams & Tasks (35 min)

> Le **Dynamic Tables** sono tabelle che si aggiornano automaticamente quando cambiano i dati sorgente. Definisci la trasformazione SQL una volta e Snowflake gestisce il refresh incrementale. Gli **Streams** catturano le modifiche (CDC) su una tabella, mentre i **Tasks** permettono di schedulare operazioni SQL. Insieme, offrono pipeline ETL declarative e event-driven.

Obiettivi di Apprendimento

- Comprendere il concetto di Dynamic Tables
- Creare pipeline di trasformazione dati automatiche
- Monitorare lo stato delle Dynamic Tables
- Utilizzare Streams e Tasks per pipeline event-driven

Istruzioni Passo-Passo

Step 4.1: Prima Dynamic Table - Ascolti Giornalieri

```
USE ROLE ACCOUNTADMIN;
USE SCHEMA MEDIASET_LAB.ANALYTICS;

CREATE OR REPLACE DYNAMIC TABLE ASCOLTI_GIORNALIERI
    TARGET_LAG = '1 hour'
    WAREHOUSE = MEDIASET_WH
AS
SELECT
    a.data_rilevazione,
    p.programma_id,
    p.titolo,
    p.genere,
    p.canale,
    SUM(a.telespettatori) as telespettatori_totali,
    AVG(a.share_percentuale) as share_medio
FROM MEDIASET_LAB.RAW.ASCOLTI a
JOIN MEDIASET_LAB.RAW.PROGRAMMI_TV p ON a.programma_id = p.programma_id
GROUP BY a.data_rilevazione, p.programma_id, p.titolo, p.genere,
p.canale;
```

Step 4.2: Dynamic Table a Cascata - Top Settimanali

```
CREATE OR REPLACE DYNAMIC TABLE TOP_PROGRAMMI_SETTIMANA
    TARGET_LAG = '1 hour'
    WAREHOUSE = MEDIASET_WH
AS
WITH ranked AS (
    SELECT
        DATE_TRUNC('week', data_rilevazione) as settimana,
        titolo,
        genere,
        AVG(share_medio) as share_medio_settimana,
        ROW_NUMBER() OVER (
            PARTITION BY DATE_TRUNC('week', data_rilevazione)
            ORDER BY AVG(share_medio) DESC
        ) as rank
    FROM ASCOLTI_GIORNALIERI
    GROUP BY DATE_TRUNC('week', data_rilevazione), titolo, genere
)
SELECT * FROM ranked WHERE rank <= 10;
```

Step 4.3: Verifica e Monitoraggio

```
-- Visualizza lo stato
SHOW DYNAMIC TABLES IN SCHEMA MEDIASET_LAB.ANALYTICS;

-- Query sulla dynamic table
SELECT * FROM TOP_PROGRAMMI_SETTIMANA
ORDER BY settimana DESC, rank;
```

Step 4.4: Simulazione Aggiornamento

```
-- Inserisci nuovi dati
INSERT INTO MEDIASET_LAB.RAW.ASCOLTI VALUES
(99999, 1, 2, CURRENT_DATE(), 'Prime Time', 'Lombardia', 3500000, 28.5,
'Adulti 25-54', 'Smart TV');

-- Forza refresh (opzionale per demo)
ALTER DYNAMIC TABLE ASCOLTI_GIORNALIERI REFRESH;

-- Verifica aggiornamento
SELECT * FROM ASCOLTI_GIORNALIERI WHERE data_rilevazione =
CURRENT_DATE();
```

Esercizio Pratico 4

Crea una Dynamic Table che calcoli i KPI pubblicitari aggregati per settore merceologico.

Streams & Tasks: Pipeline Event-Driven

> Gli **Streams** tracciano le modifiche (INSERT, UPDATE, DELETE) su una tabella sorgente tramite Change Data Capture (CDC). I **Tasks** eseguono istruzioni SQL su base schedulata o quando uno stream contiene nuovi dati. Combinati, permettono di costruire pipeline reattive senza orchestrazione esterna.

Step 4.5: Creazione di uno Stream sulla Tabella ASCOLTI

```
USE ROLE ACCOUNTADMIN;
USE SCHEMA MEDIASET_LAB.RAW;

-- Crea uno stream per catturare le modifiche sulla tabella ASCOLTI
CREATE OR REPLACE STREAM ASCOLTI_STREAM ON TABLE ASCOLTI
    APPEND_ONLY = TRUE;

-- Verifica: lo stream è vuoto (nessuna modifica ancora)
SELECT * FROM ASCOLTI_STREAM;
```

Nota: `APPEND_ONLY = TRUE` cattura solo gli INSERT, ideale per dati di tipo log/eventi come gli ascolti TV.

Step 4.6: Tabella Target e Task per Processare lo Stream

```
USE SCHEMA MEDIASET_LAB.ANALYTICS;

-- Crea una tabella target per i nuovi ascolti processati
CREATE OR REPLACE TABLE NUOVI_ASCOLTI_LOG (
    data_elaborazione TIMESTAMP DEFAULT CURRENT_TIMESTAMP(),
    programma_id NUMBER,
    data_rilevazione DATE,
    regione VARCHAR(50),
    telespettatori NUMBER,
    share_percentuale FLOAT,
    num_record_processati NUMBER
);

-- Crea un task che consuma lo stream e scrive nella tabella target
CREATE OR REPLACE TASK PROCESSA_NUOVI_ASCOLTI
    WAREHOUSE = MEDIASET_WH
    SCHEDULE = '5 MINUTE'
    WHEN SYSTEM$STREAM_HAS_DATA('MEDIASET_LAB.RAW.ASCOLTI_STREAM')
AS
INSERT INTO MEDIASET_LAB.ANALYTICS.NUOVI_ASCOLTI_LOG (
    programma_id, data_rilevazione, regione,
    telespettatori, share_percentuale, num_record_processati
)
SELECT
    programma_id,
    data_rilevazione,
    regione,
    SUM(telespettatori),
    AVG(share_percentuale),
    COUNT(*)
FROM MEDIASET_LAB.RAW.ASCOLTI_STREAM
GROUP BY programma_id, data_rilevazione, regione;

-- Attiva il task
ALTER TASK PROCESSA_NUOVI_ASCOLTI RESUME;

-- Verifica stato del task
SHOW TASKS IN SCHEMA MEDIASET_LAB.ANALYTICS;
```

Nota: La clausola `WHEN SYSTEM$STREAM_HAS_DATA(...)` fa sì che il task si esegua solo quando ci sono nuovi dati nello stream, risparmiando risorse.

Step 4.7: Test della Pipeline Stream + Task

```
-- Inserisci nuovi dati nella tabella sorgente
INSERT INTO MEDIASET_LAB.RAW.ASCOLTI VALUES
(99801, 3, 5, CURRENT_DATE(), 'Access Prime Time', 'Lombardia', 2100000,
19.3, 'Adulti 25-54', 'Smart TV'),
(99802, 3, 5, CURRENT_DATE(), 'Access Prime Time', 'Lazio', 1500000,
16.8, 'Adulti 25-54', 'TV Tradizionale');

-- Verifica che lo stream ha catturato le modifiche
SELECT * FROM MEDIASET_LAB.RAW.ASCOLTI_STREAM;

-- Esegui il task manualmente (senza aspettare lo schedule)
EXECUTE TASK PROCESSA_NUOVI_ASCOLTI;

-- Verifica i risultati nella tabella target
SELECT * FROM MEDIASET_LAB.ANALYTICS.NUOVI_ASCOLTI_LOG
ORDER BY data_elaborazione DESC;

-- Lo stream è ora vuoto (i dati sono stati consumati)
SELECT * FROM MEDIASET_LAB.RAW.ASCOLTI_STREAM;
```

Step 4.8: Pulizia Tasks

```
-- Sospendi il task quando non serve più (best practice)
ALTER TASK PROCESSA_NUOVI_ASCOLTI SUSPEND;
```

Esercizio Pratico 5

Crea uno stream sulla tabella `FEEDBACK_SOCIAL` e un task che, quando arrivano nuovi feedback, inserisca un riepilogo aggregato per `sentiment` in una nuova tabella `ANALYTICS.FEEDBACK_RIEPILOGO`.

Riepilogo e Best Practices

Cosa Abbiamo Imparato

1. **Snowflake Basics**: Database, Schema, Warehouse, Ruoli
2. **Zero Copy Cloning e Time Travel**: Clonazione istantanea e recupero dati storici
3. **Security**: Masking Policy, Row Access Policy

4. **Pipelines**: Dynamic Tables per ETL automatico, Streams & Tasks per pipeline event-driven

Best Practices

- Usa ruoli specifici per ogni tipo di utente
- Implementa masking su tutti i dati sensibili (PII)
- Preferisci Dynamic Tables per trasformazioni ricorrenti
- Usa Zero Copy Cloning per ambienti di test/dev senza costi di storage
- Abilita Time Travel per proteggere i dati da cancellazioni accidentali
- Sospendi sempre i Tasks quando non sono necessari per risparmiare crediti

Risorse Utili

- Documentazione Snowflake
- Snowflake Community

Cleanup (Opzionale)

Se desideri rimuovere tutti gli oggetti creati:

```
USE ROLE ACCOUNTADMIN;

-- Rimuovi il database (elimina tutto)
DROP DATABASE IF EXISTS MEDIASET_LAB;

-- Rimuovi i ruoli
DROP ROLE IF EXISTS MEDIASET_ADMIN;
DROP ROLE IF EXISTS MEDIASET_ANALYST;
DROP ROLE IF EXISTS MEDIASET_MARKETING;
DROP ROLE IF EXISTS MEDIASET_REGIONALE_NORD;
DROP ROLE IF EXISTS MEDIASET_REGIONALE_SUD;

-- Rimuovi il warehouse
DROP WAREHOUSE IF EXISTS MEDIASET_WH;
```

Grazie per aver partecipato al workshop!

Per domande o supporto: [inserire contatto]